

Problem 1.b - Training and Validation with 5 Explanatory Variables

Importing `train_test_split` from `sklearn.model_selection` I split the housing data into a training set 80% of the original size, and put the remaining 20% into a validation set. Due to the high variation in explanatory variables, overflow occurred and the cost function for both the training and validation sets. This is later fixed with normalization and standardization. The cost function can be seen in Figure 1.a.

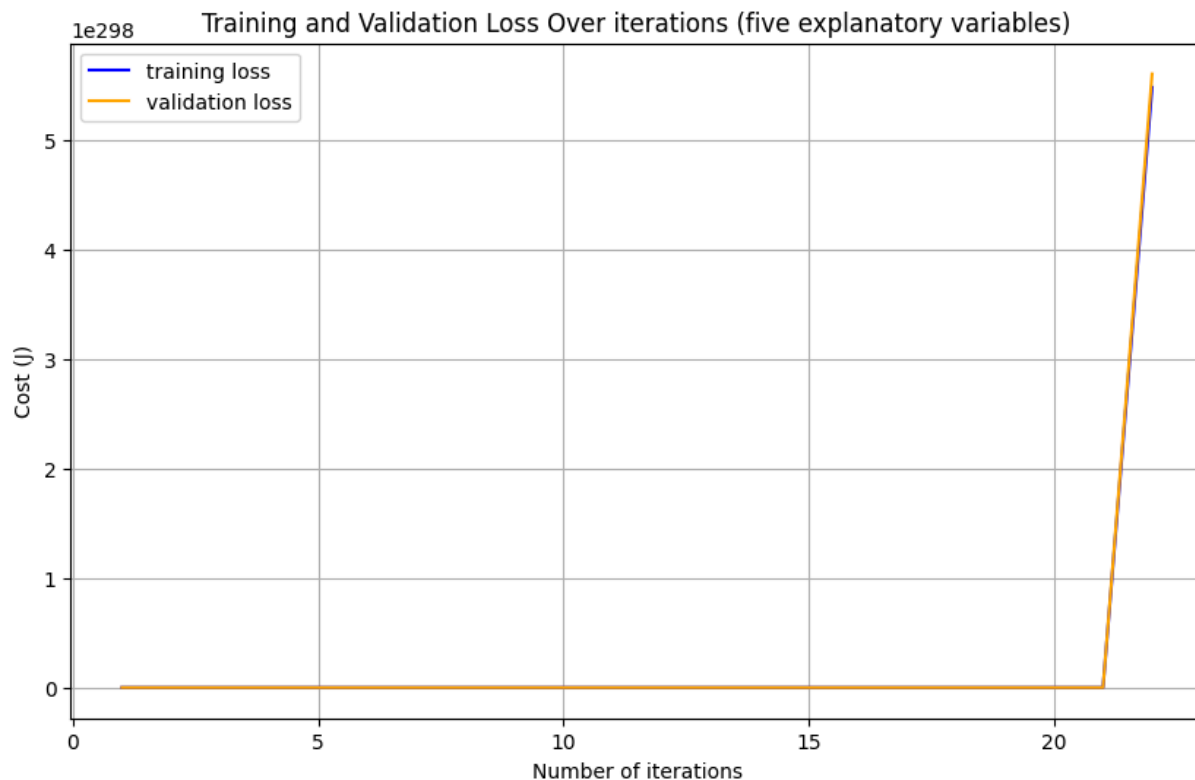


Figure 1.a - Cost function of training and validation sets without preprocessing for 5 explanatory variables.

The following output shows the overflow that occurred due to the lack of preprocessing.

```
RuntimeWarning: overflow encountered in square
sqrErrors = np.square(errors)
/tmp/ipykernel_1112/2089373709.py:66: RuntimeWarning: invalid value encountered in subtract
theta = theta - sum_delta;
Final value of theta_housing_five_vars_training = [nan nan nan nan nan nan]
cost_history of housing_five_vars_training = [1.11635150e+26 1.07911858e+39
1.04312747e+52 ... nan nan nan]
cost_history of housing_five_vars_validation = [1.14112516e+26 1.10306596e+39
1.06627614e+52 ... nan nan nan]
```

Problem 1.b - Training and Validation with 11 Explanatory Variables

Like in problem 1 part a, overflow occurred even with the addition of 6 more explanatory variables. This is due the large raw values of price and area produced really big gradients, which caused the gradient to overshoot the minimum until theta grew large enough to exceed the 64-bit size of a float. The cost function can be seen in Figure1.b

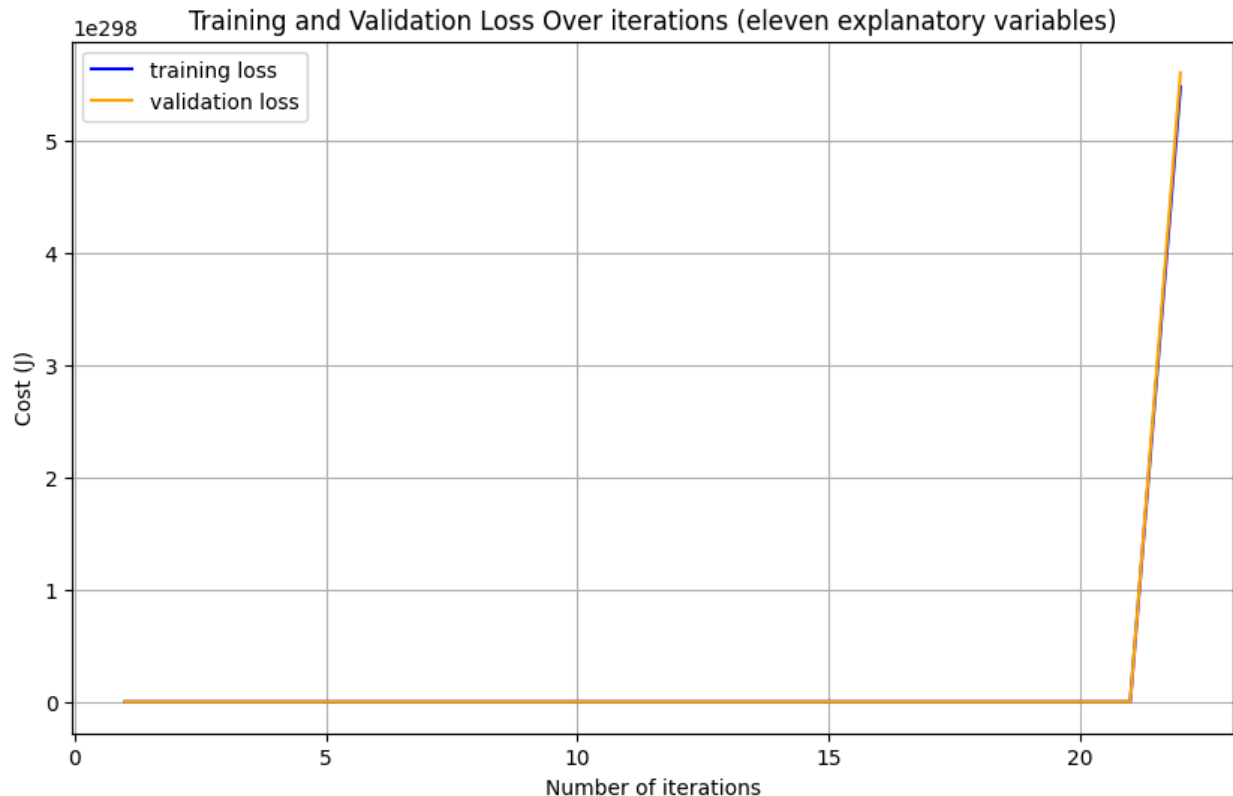


Figure 1.b - Cost function of training and validation sets without preprocessing for 11 explanatory variables.

The following output shows the overflow that occurred due to the lack of preprocessing.

```
RuntimeWarning: overflow encountered in square
sqrErrors = np.square(errors) /tmp/ipykernel_1112/1278459152.py:72: RuntimeWarning: invalid
value encountered in subtract
theta = theta - sum_delta;
Final value of theta housing_eleven_vars_training = [nan nan nan nan nan nan nan nan nan
nan nan nan]
cost_history of housing_eleven_vars_training = [1.11635158e+26 1.07911873e+39
1.04312768e+52 ...nan nan nan]
cost_history of housing_eleven_vars_validation = [1.14112524e+26 1.10306611e+39
1.06627636e+52 ... nan nan nan]
```

Problem 2.a - Normalization and Standardization

Importing MinMaxScaler from sklearn.preprocessing we can select the appropriate explanatory variables (5 for part a and 11 for part b) and fit_transform the training data and .transform the validation data. This is because we need to fit the min/max values to the training data and use that fitment in the transformation for the validation set. This way the values scale the same. The cost function of the training set and validation set with normalization for 5 explanatory variables can be seen in Figure 2.a.

The output, however, was not normalized so that we can conceptualize the loss in dollars. The cost history is in dollars squared from the cost function. So taking the square root of $7.87069728 \times 10^{11}$ gives us roughly \$887,169.50 of inaccuracy.

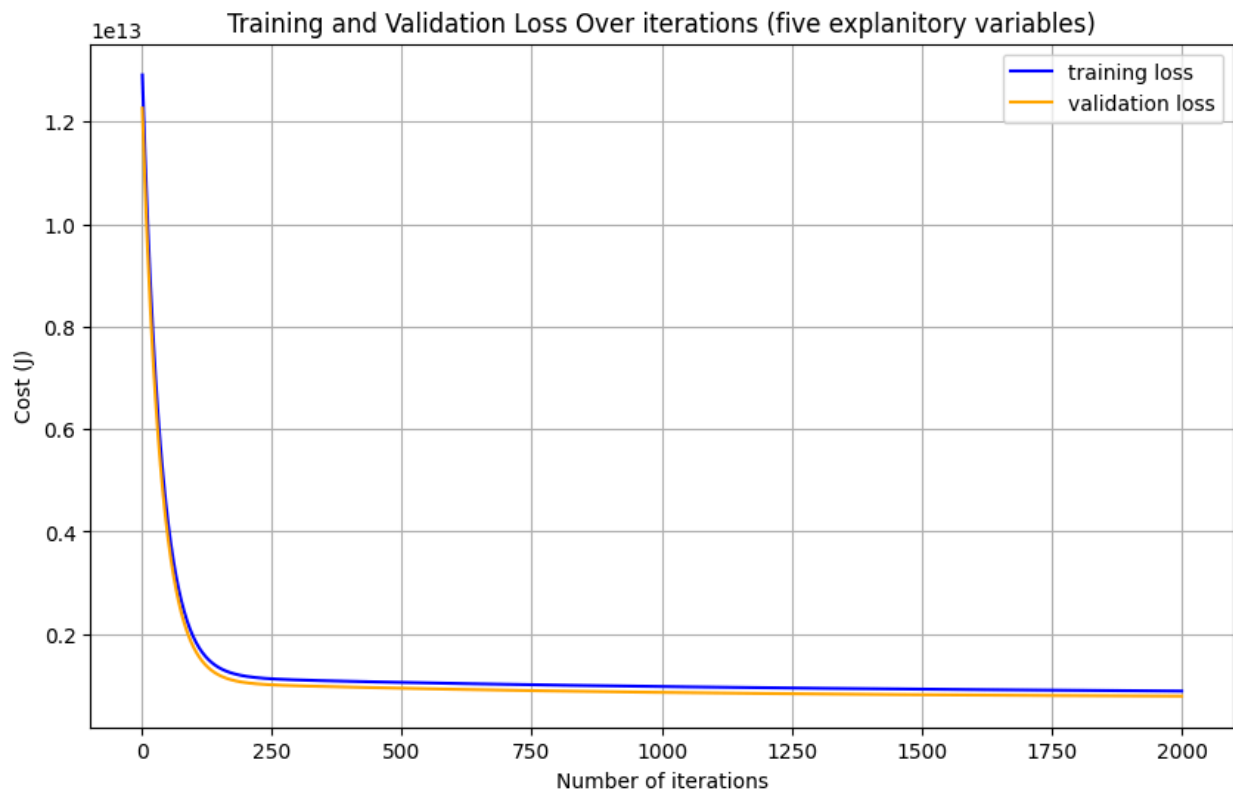


Figure 2.a.1 - Cost function of training and validation sets with normalization for 5 explanatory variables.

Final value of theta housing_five_vars_training = [2748676.7809976083

2241568.182180497 1403394.608289216 1890740.4723930121 1700157.82121759
1465097.767312941]

cost_history of housing_five_vars_training = [1.29111285e+13 1.25923490e+13
1.22822200e+13 ... 8.86749897e+11 8.86689176e+11 8.86628494e+11]

cost_history of housing_five_vars_validation = [1.22628811e+13 1.19550255e+13
1.16555795e+13 ... 7.87151267e+11 7.87110476e+11 7.87069728e+11]

To use standardization instead of normalization, we can simply import StandardScaler in place of MinMaxScaler. This gives us an observable decrease in loss compared to normalization.

Again, when standardizing everything except the price, we can gain perspective on how much loss there actually is. It can be observed in Figure 2.a.2 that standardization provides a lower cost for the training set, but has a greater cost for the validation set when compared to normalization. With the validation set being the key marker of the model's performance, it poses a question on which method of preprocessing should be used and goes against my initial assumption that standardization provided more accuracy than normalization.

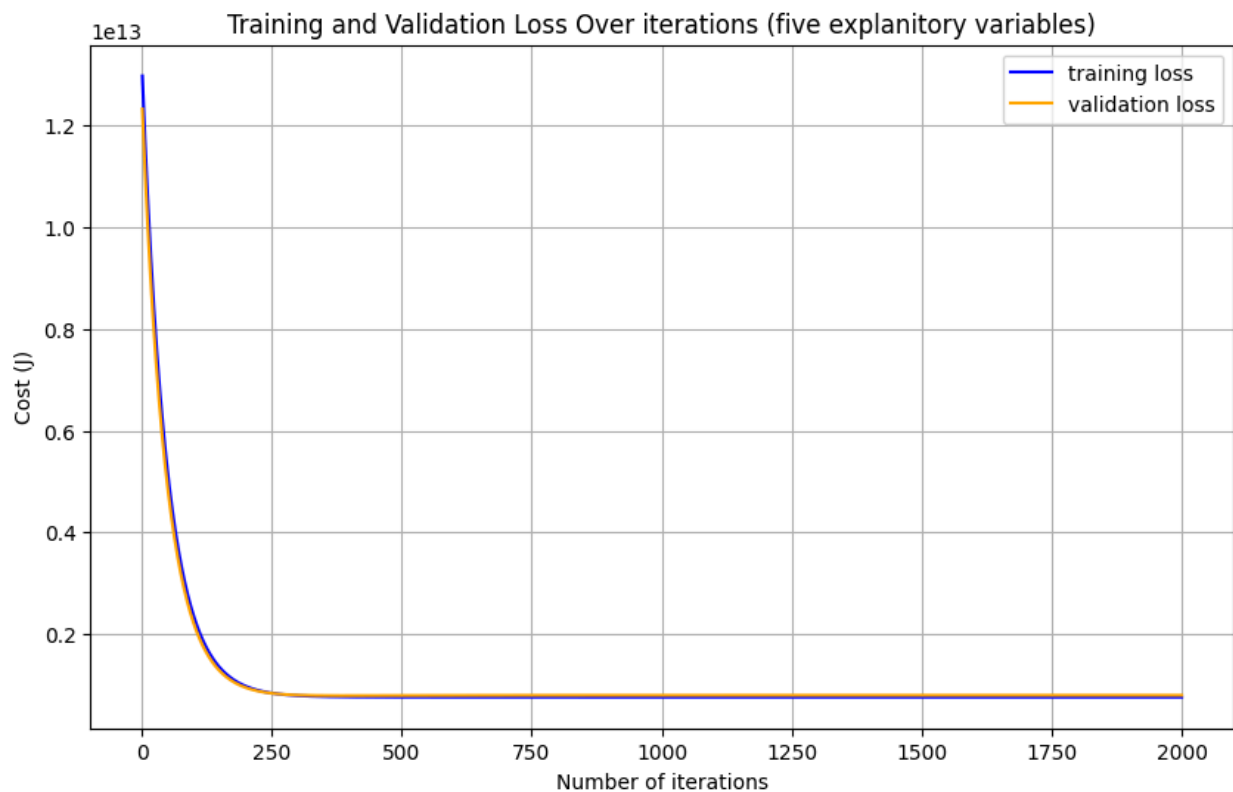


Figure 2.a.2 - Cost function of training and validation sets with standardization for 5 explanatory variables.

Final value of theta housing_five_vars_training = [4795729.211245418 729932.491706186 79337.82792296058 641263.7384777205 463853.7014507087 287330.7122600231]
cost_history of housing_five_vars_training = [1.29761832e+13 1.27193416e+13 1.24681395e+13 ... 7.61034841e+11 7.61034841e+11 7.61034841e+11]
cost_history of housing_five_vars_validation = [1.23270247e+13 1.20802801e+13 1.18390151e+13 ... 7.98266808e+11 7.98266811e+11 7.98266813e+11]

Problem 2.b - Results

The more explanatory variables we add, the lower the loss becomes. This is because when considering more elements, we get more data on what factors drive house pricing.

Naturally, loss is lower when using 11 features instead of the previous 5. Otherwise, we can observe the same behavior discussed in problem 2.a. Standardization provides a lower loss for training but a little higher cost for validation. The cost function for normalization can be seen in Figure 2.b.1.

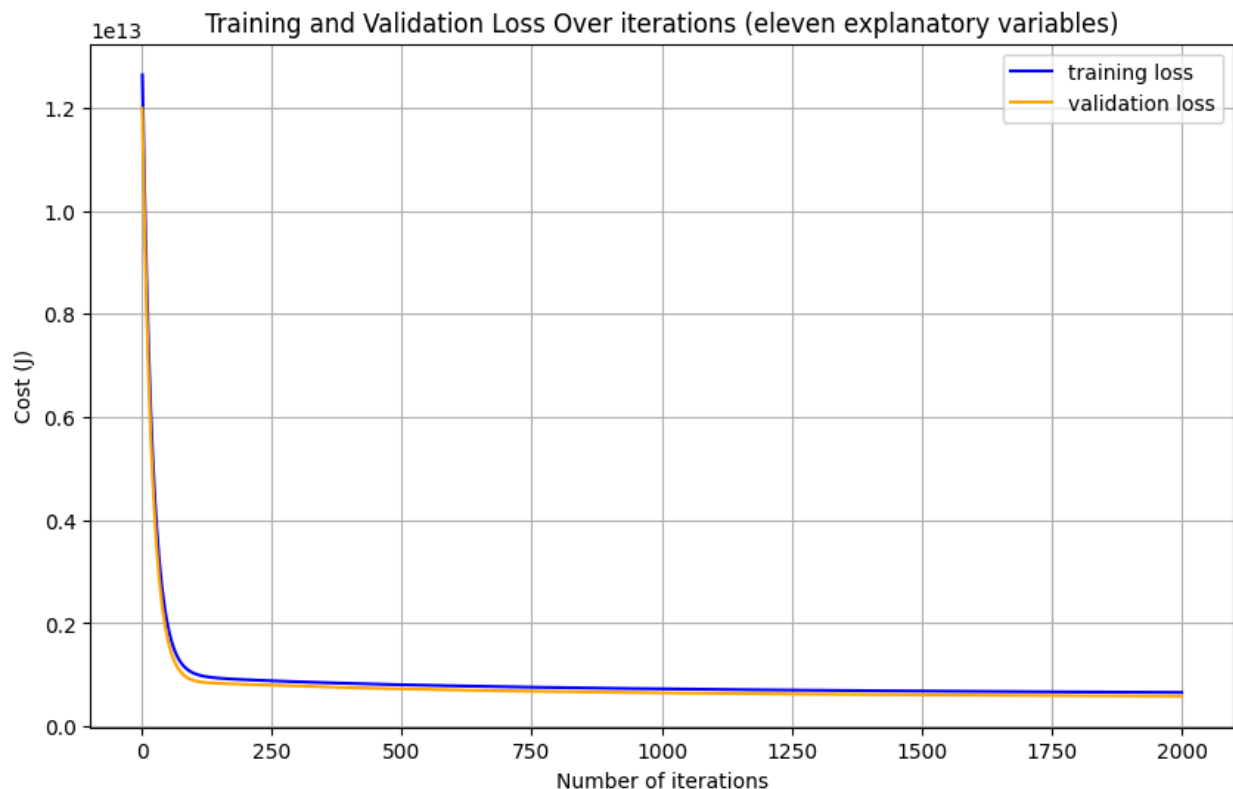


Figure 2.b.1 - Cost function of training and validation sets with normalization for 11 explanatory variables.

```
Final value of theta housing_eleven_vars_training = [1753036.1628927304
1477321.9256610707 1092493.7287270555 1659508.9191270766 1326630.1849430336
910408.1608502462 541587.6652615988 316357.6072444417 714508.6920482902
1055414.660049396 1054764.8861816374 664130.1744855446]
cost_history of housing_eleven_vars_training = [1.26325860e+13 1.20563818e+13
1.15087000e+13 ... 6.59268577e+11 6.59224936e+11 6.59181328e+11]
cost_history of housing_eleven_vars_validation = [1.19851087e+13 1.14206570e+13
1.08845392e+13 ... 5.87959496e+11 5.87923099e+11 5.87886738e+11]
```

With standardization instead of normalization, we can simply import StandardScaler in place of MinMaxScaler. The cost function using standardization can be seen in Figure 2.b.2.

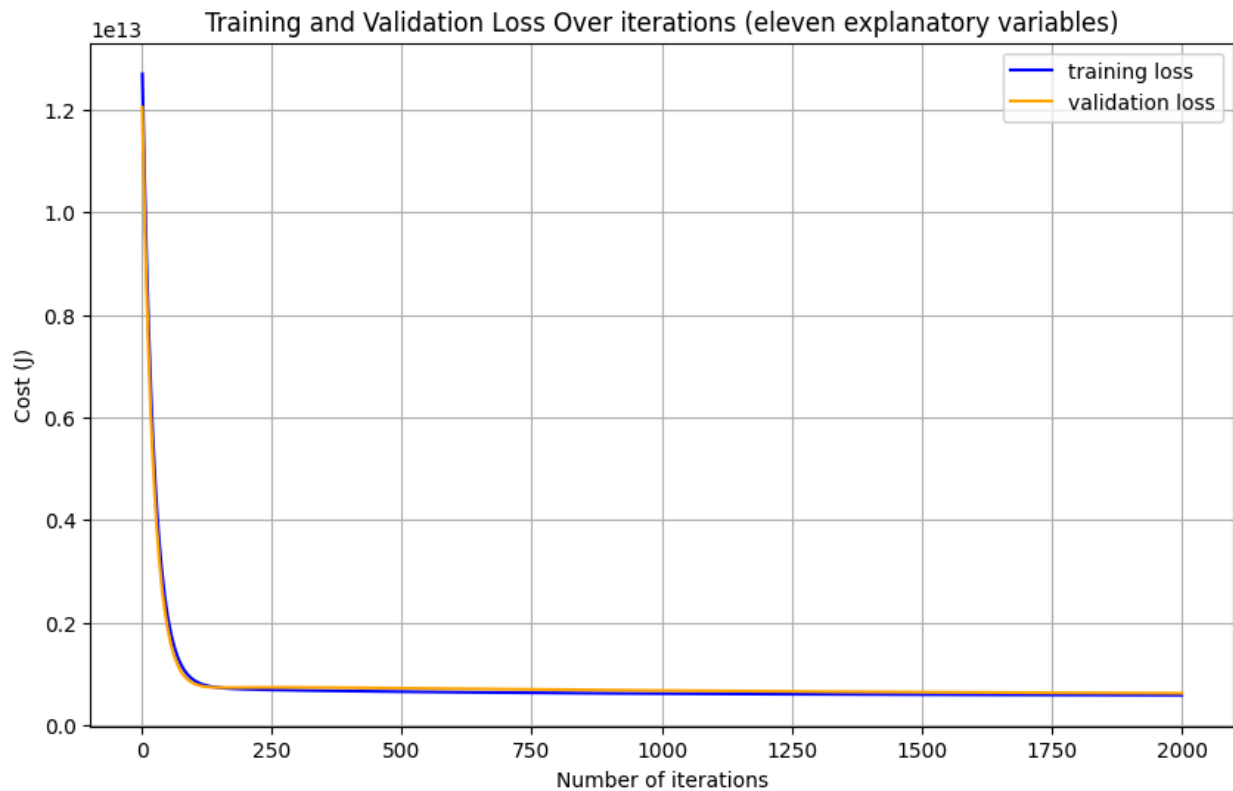


Figure 2.b.2 - Cost function of training and validation sets with standardization for 11 explanatory variables.

Final value of theta housing_eleven_vars_training = [3206921.7467073444
482667.5149263145 88145.04283695675 594603.6468751007 342470.96920909797
1091330.2132073427 361004.2205325065 348202.734595767 648191.745843577
904523.6251072377 205656.041225984 561734.5847513857]
cost_history of housing_eleven_vars_training = [1.26967614e+13 1.21782846e+13
1.16823433e+13 ... 5.88318375e+11 5.88302329e+11 5.88286301e+11]
cost_history of housing_eleven_vars_validation = [1.20483668e+13 1.15408408e+13
1.10557721e+13 ... 6.24991026e+11 6.24956475e+11 6.24921952e+11]

Problem 3.a - Results

When applying regularization to the 5 feature test, there was minimal to no observable change in the observed cost for all tests. Lambda values ranging from 0.01 to 10 were used in testing. I settled on the value of 0.01. Part b is a different story. Regardless, standardization was used across the board as it provided the lowest cost when compared to normalization.

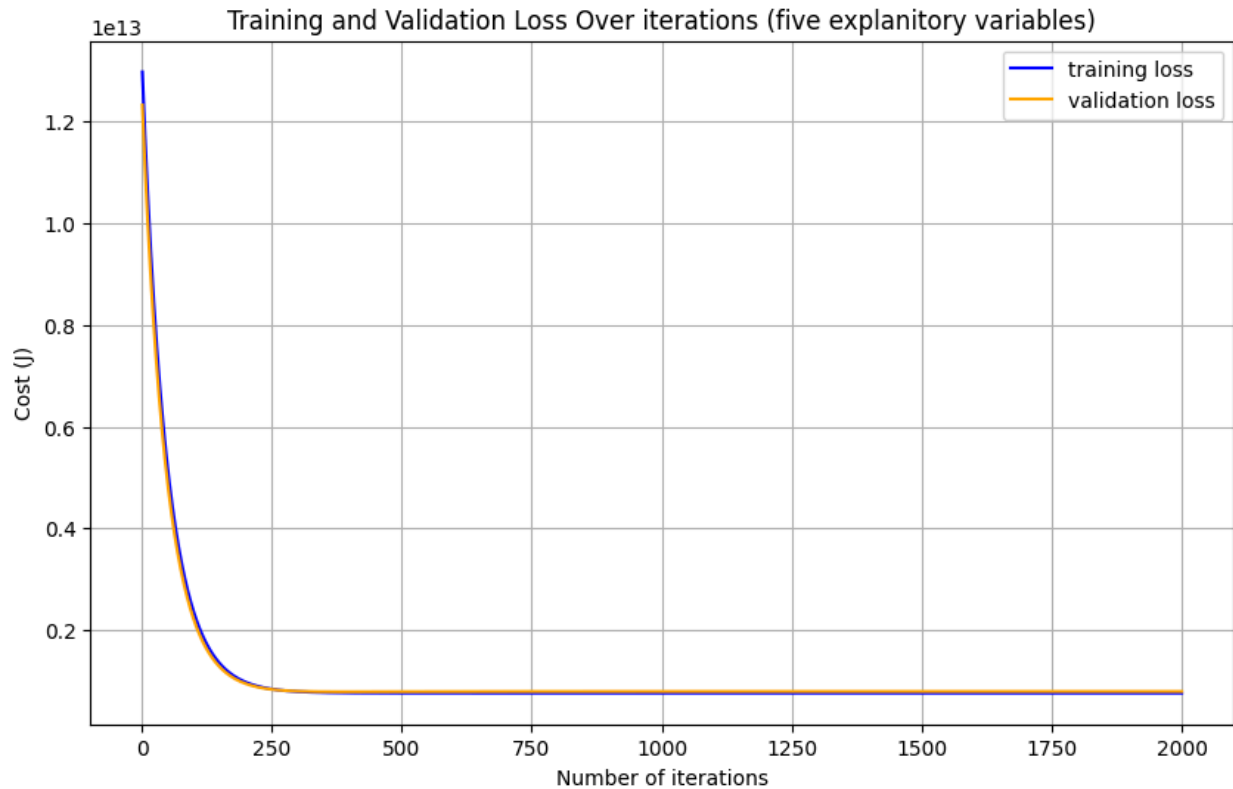


Figure 3.a.1 - Cost function of training and validation sets with regularization and standardization for 5 explanatory variables.

For lambda = 0.01:

Final value of theta housing_five_vars_training = [4795729.211245417 715556.5525494925 87560.79234303713 629473.622747028 454862.8930435852 286443.26060171245]
cost_history of housing_five_vars_training = [1.29761832e+13 1.27193492e+13 1.24681614e+13 ... 7.61279502e+11 7.61279502e+11 7.61279502e+11]
cost_history of housing_five_vars_validation = [1.23270247e+13 1.20802869e+13 1.18390348e+13 ... 7.94608942e+11 7.94608944e+11 7.94608945e+11]

For lambda = 1:

Final value of theta housing_five_vars_training = [4795729.211245417 729929.115930746 79339.81718246617 641260.9653920018 463851.57661533274 287330.5283334395]

cost_history of housing_five_vars_training = [1.29761832e+13 1.27193416e+13 1.24681395e+13 ... 7.61034841e+11 7.61034841e+11 7.61034841e+11]

cost_history of housing_five_vars_validation = [1.23270247e+13 1.20802801e+13 1.18390151e+13 ... 7.98265877e+11 7.98265880e+11 7.98265883e+11]

For lambda = 5:

Final value of theta housing_five_vars_training = [4795729.211245418 729915.6131990674 79347.77372679232 641249.8733754468 463843.0775726188 287329.79253267916]

cost_history of housing_five_vars_training = [1.29761832e+13 1.27193416e+13 1.24681395e+13 ... 7.61034842e+11 7.61034842e+11 7.61034842e+11]

cost_history of housing_five_vars_validation = [1.23270247e+13 1.20802801e+13 1.18390151e+13 ... 7.98262155e+11 7.98262157e+11 7.98262160e+11]

For lambda = 10:

Final value of theta housing_five_vars_training = [4795729.211245417 729898.7356171028 79357.71829649027 641236.0090889157 463832.45444144687 287328.8725692965]

cost_history of housing_five_vars_training = [1.29761832e+13 1.27193416e+13 1.24681395e+13 ... 7.61034843e+11 7.61034843e+11 7.61034843e+11]

cost_history of housing_five_vars_validation = [1.23270247e+13 1.20802801e+13 1.18390152e+13 ... 7.98257502e+11 7.98257505e+11 7.98257507e+11]

Problem 3.b - Results

Regularization minimally lowers the cost when compared to problem 2.b. Lowering the lambda continued to lower the cost, a value of $\lambda = 0.01$ was used here.

For $\lambda = 0.03$:

Final value of theta housing_eleven_vars_training = [3206983.747509175
482673.10051976325 88145.4848815876 594600.4915890787 342475.14216498187
1091274.4955917853 360998.3518672677 348201.8373470485 648149.1045391201
904495.1983258933 205659.89460476325 561723.3399267498]
cost_history of housing_eleven_vars_training = [1.26967614e+13 1.21782846e+13
1.16823434e+13 ... 5.88315887e+11 5.88299843e+11 5.88283816e+11]
cost_history of housing_eleven_vars_validation = [1.20483668e+13 1.15408408e+13
1.10557722e+13 ... 6.24982895e+11 6.24948346e+11 6.24913824e+11]

For $\lambda = 0.01$:

Final value of theta housing_eleven_vars_training = [3206942.414049782
482669.37684833666 88145.19017036784 594602.5951039412 342472.3602499734
1091311.6402419405 361002.26430098317 348202.4355770822 648177.5316577783
904514.1493296111 205657.3257216462 561730.8364443511]
cost_history of housing_eleven_vars_training = [1.26967614e+13 1.21782846e+13
1.16823434e+13 ... 5.88317545e+11 5.88301500e+11 5.88285473e+11]
cost_history of housing_eleven_vars_validation = [1.20483668e+13 1.15408408e+13
1.10557721e+13 ... 6.24988315e+11 6.24953765e+11 6.24919242e+11]

